

datalab 报告

姓名：丛文博

学号：2025201897

题目	得分
bitAnd	1/1
bitXor	1/1
samesign	2/2
logtwo	4/4
byteSwap	4/4
reverse	3/3
logicalShift	3/3
leftBitCount	4/4
float_i2f	4/4
floatScale2	4/4
float64_f2i	3/3
floatPower2	4/4
总分	37/37

test 截图：

```
cccrb@WSL: /home/cccrb/datalab2026
$ cd /home/cccrb/datalab2026
$ python3 test.py -V
bitAnd      1/1: PASS    detail: Pass using 4 operations including {'|', '~'} and {'int'}.
bitXor      1/1: PASS    detail: Pass using 7 operations including {'~', '&'} and {'int'}.
samesign    2/2: PASS    detail: Pass using 7 operations including {'^', '|', '>>'} and {'if', 'int'}.
logtwo      4/4: PASS    detail: Pass using 24 operations including {'|', '>', '<<', '>>'} and {'int'}.
byteSwap    4/4: PASS    detail: Pass using 11 operations including {'^', '&', '<<', '+', '>>'} and {'int'}.
reverse     3/3: PASS    detail: Pass using 23 operations including {'|', '&', '>>', '<<'} and {'unsigned'}.
logicalShift 3/3: PASS    detail: Pass using 3 operations including {'&', '>>'} and {'int'}.
leftBitCount 4/4: PASS    detail: Pass using 34 operations including {'|', '<<', '+', '>>', '~'} and {'int'}.
float_i2f    4/4: PASS    detail: Pass using 27 operations including {'|', '&', '<<', '>', '+', '-', '<', '>>', '|', '==', '~'} and {'if', 'while', 'unsigned', 'int'}.
floatScale2  4/4: PASS    detail: Pass using 17 operations including {'&', '<<', '+', '<', '|', '==', '>='} and {'if', 'unsigned', 'int'}.
float64_f2i  3/3: PASS    detail: Pass using 18 operations including {'&', '<<', '<=', '-', '+', '>>', '<', '|', '~'} and {'if', 'int', 'unsigned'}.
floatPower2  4/4: PASS    detail: Pass using 9 operations including {'-', '<<', '+', '>='} and {'if', 'unsigned', 'int'}.
Total points: 37
```

解题报告

亮点

1. `byteSwap` —— 只算两个字节的差异位，一次异或同时换掉两个字节
2. `logicalShift` —— 用000.....111这样的掩码以及&号实现左边补全的位全为0
3. `float_i2f` —— 将 $\sim x + 1 = x$ 的0x80000000单独拎出来然后先移掉符号位并把cnt初始值定为1
4. `float64_f2i` —— 把可能的右移32拆成31+1避免违规，并且分类得当

bitAnd

交为补并补的补

```
return ~(~x|~y);
```

bitXor

即 $(x|y) \& \sim(x \& y)$ 然后并为补交补的补

```
return ~(x&y)&~(~x&~y);
```

samesign

只要有一个0就单独考虑 否则直接比较符号位

```
return !(x>>31 ^ y>>31);
```

logtwo

二分法判断前半一半有没有数字 比如这个判断前16位

```
cnt=cnt|((v>0xFFFF)<<4); v=v>>((v>0xFFFF)<<4);
```

后面依次 8、4、2、1，5 步覆盖 0~31。

byteSwap

本来是分别把两个byte提出再填入的，但是最大符号数量不够，于是计算差值，最后相当于交换

```
int k=s^t;
x=((k<<M)+(k<<N))^x;
```

reverse

分治，先按 1 位一组两两对调，再 2、4、8、16，于是完全对称过来了

```
v=((v>>1)&0x55555555)|((v&0x55555555)<<1);
```

logicalShift

同“亮点”用000.....111这样的掩码以及&号实现左边补全的位全为0

```
x=(x>>n)&(0xFFFFFFFF>>n);
```

leftBitCount

受log2二分法的启发，但是5步只有 $2^5=32$ 种结局，但答案有33种，所以末尾还要再补一次符号位：

```
cnt=cnt+!(~(x>>31));
```

float_i2f

根据题意一步步来，把符号阶码尾数算准确，最后再“四舍五入”（其实并非四舍五入 但是按题意要求即可）

`INT_MIN` 单独拎出来是因为 `~x+1` 对它溢出，其余情况先移掉符号位，把移位次数 `cnt` 从 1 起算，最后 `31-cnt+127` 一步出阶码。

舍入是**就近取偶**，丢掉的 8 位正好等于 `0x80` 时看保留部分末位的奇偶：

```
return (t|((31-cnt+127)<<23))+X;
```

最后拼的时候用 `+` 不用 `|`，因为尾数万一从全 1 进位到 `0x800000`，这一位正好是阶码的最低位，只有 `+` 加得上去。

floatScale2

拆出符号位阶码尾数然后看阶码，如果是规格化数就阶码+1，如果不是就尾数左移，注意左移成规格化数以及左右大边界

float64_f2i

在草稿本上画出 $32+32=1+11+52$ ，先把这个64位浮点数搞清楚再将其转化为int，即float_i2f的反向思路，分情况是因为int要保留的位可能分布在uf1和uf2，然后注意自己算的是绝对值，不要弄错符号即可

floatPower2

x “相当于” 阶码，如果太小就是0，次小为非规格化数，再就是规格化，太大是上限

反馈/收获/感悟/总结

学会了对遇到的问题进行拆分（分情况讨论或者分步骤比如右移32拆成31+1）以及对问题进行等效处理 然后用纸笔打草稿真的很方便

参考的重要资料

1. 《深入理解计算机系统》原书第 3 版，第 2 章「信息的表示和处理」
2. IEEE 754-2019 浮点运算标准
3. 课程 PPT